

Lab 3: Architectural Pattern Selection and Justification

Scenario: Multi Vendor Artisan E Commerce Marketplace (Problem Statement 31)

NAME: Diya R Gowda

SECTION: 5'C'

SRN: PES1UG24CS159

I was assigned the Microservices Architecture for the Multi Vendor Artisan E Commerce Marketplace System. The platform brings together independent vendors, each with their own storefront, product listings and payout requirements, so the system needs components that can be built, deployed and scaled on their own rather than as one large application.

Reason 1: Independent Scaling per Business Capability

Usage across the marketplace is very uneven. The Product Catalog Component is hit constantly by shoppers browsing high resolution vendor images, while the Payment Service Component is only called briefly at checkout. Microservices allow each component, Catalog, Order Manager, Payment and Vendor Account, to be scaled on its own schedule. Extra Catalog instances can be added during a sale without spending money scaling the Payment or Vendor Account services, which see far less traffic.

Reason 2: Fault Isolation Across Vendors and Functions

Artisan vendors rely on this platform for their income, so one failing part of the system should not bring down everything else. With microservices, if the Payment Service Component has an outage or the card gateway responds slowly, shoppers can still open the app, browse the Product Catalog Component and build an order. Only checkout is affected. In a single combined application, a fault in one area is far more likely to take the entire storefront offline for every vendor at once.

Security Advantage

Keeping the Payment Service Component as its own service means cardholder data and the logic that splits payouts between vendors and the platform can be locked down separately from the rest of the system, with its own access controls and its own data store. The Product Catalog and Order Manager components never touch payment credentials directly. They only talk to the Payment Service through the defined PaymentInterface, so a weakness anywhere else in the platform cannot reach payment data.

Performance Benefit

Because the Product Catalog Component is separate from the rest of the system, it can be paired with a content delivery network for vendor product images without that decision affecting checkout performance at all. Catalog browsing stays fast even while Order Manager and Payment Service are under heavy load during a checkout rush, since the read heavy catalog traffic and the write heavy order and payment traffic are handled by services that scale independently instead of competing for the same resources.